

SkVM: Revisiting Language VM for Skills across Heterogenous LLMs and Harnesses

Le Chen
Shanghai Jiao Tong University
Shanghai, China

Yubin Xia
Shanghai Jiao Tong University
Shanghai, China

Erhu Feng
Shanghai Jiao Tong University
Shanghai, China

Haibo Chen
Shanghai Jiao Tong University
Shanghai, China

Abstract

LLM agents increasingly adopt skills as a reusable unit of composition. While skills are shared across diverse agent platforms, current systems treat them as raw context, causing the same skill to behave inconsistently for different agents. This fragility undermines skill portability and execution efficiency.

To address this challenge, we analyze 118,000 skills and draw inspiration from traditional compiler design. We treat skills as code and LLMs as heterogeneous processors. To make portability actionable, we decompose a skill’s requirements into a set of primitive capabilities, and measure how well each model-harness pair supports them. Based on these capability profiles, we propose SkVM, a compilation and runtime system designed for portable and efficient skill execution. At compile time, SkVM performs capability-based compilation, environment binding, and concurrency extraction. At runtime, SkVM applies JIT code solidification and adaptive recompilation for performance optimization.

We evaluate SkVM across eight LLMs of varying scales and three agent harnesses, covering SkillsBench and representative skill tasks. Results demonstrate that SkVM significantly improves task completion rates across different models and environments while reducing token consumption by up to 40%. In terms of performance, SkVM achieves up to 3.2× speedup with enhanced parallelism, and 19–50× latency reduction through code solidification.

1 Introduction

LLM-based agents [1, 11, 36–38, 54] are reshaping how software tasks get done. Instead of writing code by hand, a developer describes a goal, and the agent reasons, invokes tools, and delivers the result [41, 58]. To support this paradigm, a growing ecosystem of skills [5, 6, 18, 50] has emerged to make agent execution more reliable and practical. Skills primarily consist of natural language descriptions and scripts, encapsulating domain-specific procedures and best practices [7]. Over 100,000 skills are now distributed across major platforms [13, 50], spanning data analysis, finance, office automation, programming, and beyond.

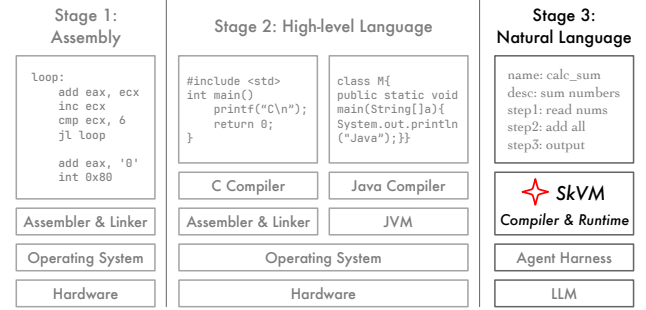


Figure 1. Evolution of programming abstractions. Skills are the current frontier: natural language programs of hundreds of lines that lack a compiler and runtime for cross-target portability.

However, current agents’ support for skills is simplistic: skills are treated as additional context and passed directly to the model. Models differ substantially in their ability to understand and execute skills [22, 30, 33]. Across eight models spanning a range of scales, enabling skills degrades performance on 15% of tasks overall (e.g., 7% for Opus 4.6 and 25% for Qwen3-30B). For another 17% of tasks, scores remain unchanged (excluding tasks with 100% completion), and on up to 87% of tasks, at least one model shows no improvement after skill usage. Recent work [22] has reached similar conclusions, finding that on the SWE-Benchmark, 39 out of 49 skills showed no score improvement, and 3 skills experienced significant score degradation.

We attribute these issues to two factors: (1) even when a skill is loaded, the model may ignore its guidance during execution; and (2) even when the model follows the skill, the skill’s assumptions about model capabilities may not match the model’s actual competence. Beyond differences in task completion gains, the semi-structured code snippets and workflow-style formats common in current skills also incur substantial token overhead (e.g., 451% token increase while pass rates remain unchanged [22]) and higher latency during execution.

This problem arises from a fundamental mismatch between static skills and the variability of underlying models and agent harnesses. The capabilities a skill demands may not align with the capabilities of the LLM invoked at runtime.

Revisiting the evolution of computing paradigms [15, 51, 53] (as shown in Figure 1), we observe that in the agent era, **skills are code**, and **LLMs are processors**. Yet today, no mechanism exists to efficiently and reliably execute skills across heterogeneous LLMs and agent harnesses.

Inspired by how classical computing systems handle code, we introduce SkVM, a compilation and runtime system for skills that enables efficient cross-model execution and provides unified underlying support for skill execution. Through SkVM, skills are compiled into forms tailored to different models, allowing each model to better understand and execute skills. Furthermore, SkVM provides a unified runtime environment that systematically manages skill loading, parsing, and concurrent execution.

We structure SkVM around classical compilation techniques: interpreted execution [25], ahead-of-time (AOT) compilation [48, 49], and just-in-time (JIT) optimization [3, 12, 15, 24, 52]. Currently, agents handle skills using only interpreted execution, feeding raw skill text directly to the model. This approach hampers skill portability and execution efficiency. In contrast, SkVM further applies AOT and JIT compilation to optimize for diverse backend models and execution environments:

AOT compilation: Skills inherently encode requirements of model capabilities, environment configuration, and parallelism opportunities. Therefore, SkVM analyzes these requirements before execution and optimizes skills accordingly. *Capability-based compilation* characterizes the capability gap between LLMs and skill requirements by extracting 26 primitive capabilities. Each capability abstracts a distinct dimension of model behavior with multiple proficiency levels. The compiler measures the target model against these capability dimensions and adapts the skill specification to better align with model strengths and limitations. Second, *environment binding* extracts implicit dependencies on packages and tools from skill descriptions and generates setup scripts executable at load time, ensuring all dependencies are satisfied before skill execution. Finally, *concurrency extraction* draws inspiration from classical compiler optimizations for data-level parallelism (DLP) [27], instruction-level parallelism (ILP) [28, 45], and thread-level parallelism (TLP) [46]. It extracts parallelism opportunities at similar three granularities from skills and explicitly exposes them to the agent harness, thereby improving task execution efficiency.

JIT optimization: At runtime, SkVM leverages runtime information to continuously optimize skill performance and correctness. *Code solidification:* Skills may contain parameterized script templates which may be instantiated multiple times during different executions. JIT compilation materializes these high-frequency templates into instantiated executable code, bypassing LLM parsing. *Adaptive recompilation:* JIT monitoring tracks model execution and adaptively recompiles skills when capability gaps emerge, ensuring skills can self-improve through iterative optimization.

After skill compilation completes, the skill runtime parses compiled skill artifacts (including optimized skills, instantiated scripts, and concurrency dependency graphs) rather than simply passing raw text to the model. Additionally, the skill runtime coordinates with the system’s available resources and tool capabilities to schedule agent execution in real time, and ensures stable and reliable agent execution.

We evaluate SkVM across eight LLMs of varying scales and three agent harnesses, covering SkillsBench [30] and representative skill tasks (118 tasks). Results demonstrate that SkVM significantly improves task completion rates (averaging 15.3%) across different models and agent harnesses [11, 38]. Furthermore, for tasks that can be completed, SkVM reduces token consumption up to 40%. In terms of performance, SkVM achieves 3.2×–50× wall-clock speedups through fine-grained parallelization and JIT code solidification.

2 Skills in the Wild

In this section, we first introduce skills, the agent that uses them, and how skills are invoked in practice. We study the skill ecosystem at scale through more than 110,000 skills collected from two major distribution platforms: clawhub.ai [13] (28,990 skills) and skills.sh [50] (89,280 skills). Based on this ecosystem analysis, we further conduct experiments to identify concrete problems in current skill usage.

2.1 Skill Definition

A *skill* is a distributable, self-contained knowledge pack that enhances an AI agent’s capability on a specific class of tasks [2, 5]. A skill may introduce knowledge the base model does not have, or constrain the model toward a prescribed workflow when it already has the relevant capability but would otherwise improvise. For example, a PDF-processing skill [6] teaches the model how to use pdfplumber for table extraction, and also constrains the model to use pypdf instead of the deprecated PyPDF2 when merging PDF files.

A skill is typically defined in a structured file comprising three layers: (1) *metadata*—a name, description, and trigger conditions that allow the agent runtime to discover and select the skill automatically; (2) *instructions*—the main body of a skill that encodes multi-step workflows, tool references, and other task-specific guidance in natural language; and (3) *bundled resources*—scripts, references, and templates stored in companion files.

This structure distinguishes skills from ad-hoc prompt snippets. The metadata makes a skill a discoverable, selectable unit, enabling the agent runtime to match tasks to applicable skills automatically rather than requiring manual prompt assembly [8]. Compared with fine-tuning, skills require no weight modification and can be operated entirely at the prompt level [32, 39], making them lightweight to distribute and easy to compose. Multiple agent platforms have converged on this abstraction [2, 8, 14, 16, 19, 36], and

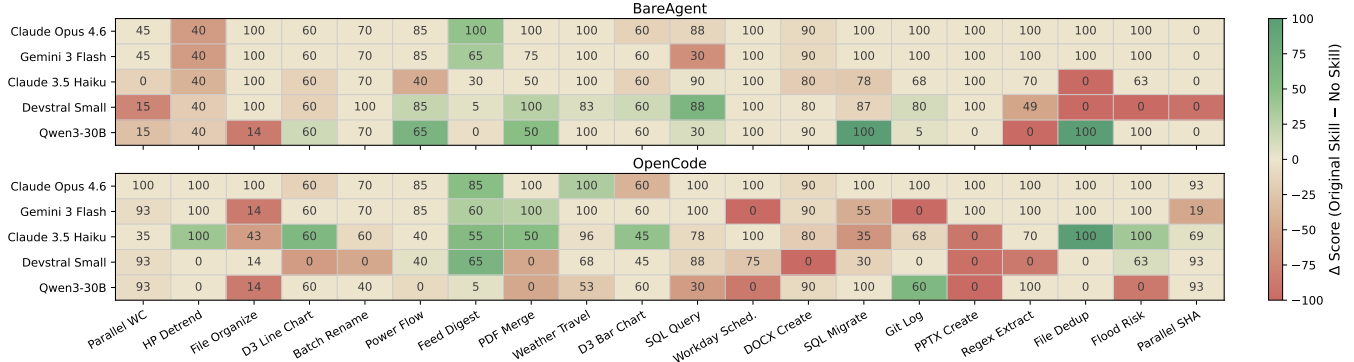


Figure 2. Skill performance across models and harnesses. Columns are tasks, rows are models, and each subplot corresponds to one harness. Cell colors indicate score deltas relative to the no-skill baseline, and cell labels show absolute task scores. Both model identity and harness choice significantly affect outcomes.

existing tools support basic skill authoring, management, and optimization.

2.2 LLM Agent and Agent Harness

An LLM agent works in a ReAct loop [58]. It receives a task, reasons about what to do, issues a tool call (reading a file, running a command, making an API request) [35, 47], observes the result, and reasons again. This loop repeats until the task is done.

The agent does not interact with the outside world directly. It runs inside an *agent harness*, a runtime framework that sits between the model and the operating environment [29, 56]. The harness manages the LLM API connection, registers the tools the agent can call, dispatches those calls, and controls how much conversation history fits in the context window. Claude Code [11], OpenCode [38], and OpenClaw [37] are examples of widely used harnesses.

Harnesses differ in ways that matter for skills [20]. Some provide a rich set of file-system and web search tools, while others expose only basic read, write, and shell execution operations. Some can spawn independent sub-agents that work concurrently, while others only support sequential execution. These differences directly affect whether a skill’s instructions can be carried out.

2.3 Characterizing Skills

Ecosystem Scale and Distribution. The skill ecosystem is large but uneven. Figure 3 shows the download counts across both platforms. On skills.sh, 89% of the 89,280 skills have fewer than 86 downloads, while a handful of head skills reach 10^4 – 10^6 . clawhub.ai follows a similar long-tailed pattern, peaking around 10^2 – 10^3 downloads. In total, over 118,000 skills exist across the two platforms, but the vast majority see little use.

Skill Taxonomy. Skills differ substantially in what they provide the agent with. We use LLM-assisted analysis to classify 15,063 skills that have more than 100 downloads into three categories, based on which aspect the skill most

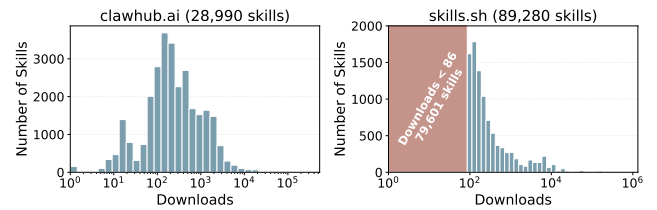


Figure 3. Skill download distribution on clawhub.ai and skills.sh. Both platforms show a long-tailed distribution.

emphasizes: tool reference (52%), procedural guidance (28%), and content generation (20%).

Tool-reference skills teach the model how to operate specific tools, APIs, or CLIs. They are essentially usage documentation loaded into the agent’s context. A representative example is a skill that documents how to use python to merge, split, and manipulate PDF files.

Procedural skills prescribe step-by-step workflows and reasoning strategies for carrying out a task. A representative example is a debugging skill that enforces a four-phase process: reproduce the failure, trace the root cause, apply a targeted fix, and verify the fix with tests.

Generative skills ask the model to produce content, code, documents, or prose. The core quality depends on the model’s own capability. An example is a frontend-design skill [6] that generates production-grade web components with specific design guidelines and styling conventions.

Tool-reference and procedural skills together account for 80% of the ecosystem. These skills encode analyzable structure, including script fragments and step-by-step workflows. The correctness of executing these skills depends on whether the agent executes the prescribed steps, not on its generative ability.

Workflow Structure. Beyond task type, skills also exhibit repeated internal structure. Across the 15,063 classified skills, 76% contain explicit procedural structure: numbered steps, conditional branches, and data dependencies between steps.

Code Fragments. 75% of skills embed code-like fragments: shell commands, API call patterns, or script snippets whose

overall shape recurs across invocations while only the input-specific parameters change. For example, a weather skill [37] may provide a family of curl command templates for current conditions, forecasts, and format options, where the command structure stays fixed while parameters such as the city, output format, and forecast day vary.

2.4 Problems with Current Skill Usage

Skills are loaded into the agent’s context as raw text, with no adaptation. But skills carry implicit assumptions about the model, agent harness, and user environment. When those assumptions are wrong, three failure modes emerge.

P1: Model Mismatch. A skill implicitly assumes that the model is capable enough to follow its instructions. In practice, models differ dramatically [33]. Figure 2 shows scores for a subset of tasks across three models and three harnesses.¹ Performance varies substantially across models on the same skill. Moreover, enabling skills degrades performance on 15% of tasks; leaves scores unchanged on 17% of tasks (excluding tasks with a 100% completion rate); and yields no improvement for at least one model on 87% of tasks.

Case Study: the pptx-creation skill [6] recommends PptxGenJS (a javascript library) for slide generation. Claude-opus-4.6 and gemini-3-flash both score 100 with this skill, but devstral-small misreads PptxGenJS as a CLI when using OpenCode and repeatedly executes wrong commands. Without the skill, devstral-small instead uses python-pptx and scores 95. The skill assumes a model can distinguish a library API from a CLI, which holds for frontier models but not weaker ones.

P2: Harness Mismatch. The same model produces different results on the same task depending on the agent harness it runs in [57]. Figure 2 also shows this dimension: across the heatmap, harness-induced variance is comparable to model-induced variance. Skills are written without knowledge of what tools and plugins the harness provides.

Case Study: Gemini 3 Flash scores 100 on the workday-scheduling task with the original skill on BareAgent, but 0 on OpenCode with the same skill. The failure is triggered by a malformed JSON key that makes the output unparseable. BareAgent adds only a minimal system prompt, whereas OpenCode prepends extensive tool documentation, and the longer combined context induces the formatting error.

P3: Environment Mismatch. Skills have dependencies on the execution environment, but the user’s machine may lack the required packages, tools, or configurations. Some skills

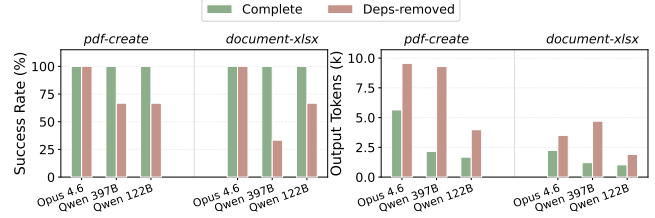


Figure 4. Removing a required dependency hurts both correctness and efficiency.

list prerequisites, but those instructions are often general and target a generic machine rather than the user’s actual setup. The real requirement depends on the current OS, hardware, installed versions, and package-manager state. Other skills omit the dependency entirely. Figure 4 shows the effect on two representative tasks. When the required library is absent, both Qwen models drop to 33–67% success rate while generating 2–4× more output tokens on failed workarounds. Even Claude Opus 4.6, which still succeeds on every run, generates 56–69% more output tokens because it must diagnose the mismatch and install the missing package at runtime. Leaving diagnosis to the model therefore turns every missing dependency into a repeated tax on both correctness and efficiency.

2.5 Challenges

Skills are distributed as plain Markdown so that any agent can load them with no special tooling. The mismatches above undermine this portability promise. Restoring portability requires adapting each skill to its target, which is challenging for two reasons.

C1: Targets Differ Along Many Axes. Each of the three dimensions above, model, harness, and environment, varies independently. A skill that needs adaptation along one dimension may need a different adaptation when two dimensions change together. The right fix for a weak model on a rich harness differs from the right fix for the same model on a minimal one. This interaction makes the adaptation space combinatorial: it cannot be covered by a fixed set of rewrite rules.

C2: Skills Are Unstructured Natural Language. Source code requires compilation before it can run on a target system, and has a grammar, types, and a well-defined AST that tools can analyze [3]. In comparison, skills are written in natural language. Although skills are mostly created following common conventions, different skills vary widely in structure, terminology, and level of detail. Before any adaptation can happen, the skill’s implicit requirements and workflow structure must be recovered from prose.

3 Design Overview

SkVM is a compilation and runtime system for skills. It makes skills portable across heterogeneous targets by combining an

¹BareAgent is a minimal agent harness that we built. It implements only basic agent runtime logic and exposes only read, write, and exec tools.

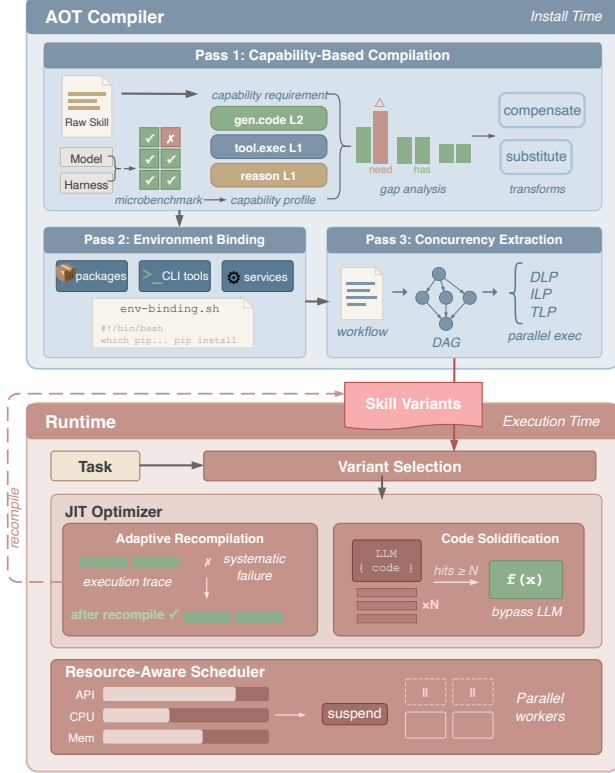


Figure 5. SkVM architecture. The AOT compiler produces optimized skill variants at install time through three passes. The runtime manages variant selection, JIT optimization, and resource-aware scheduling during execution.

AOT compiler [3] that specializes skills at install time with a runtime that resolves execution-time uncertainty. Figure 5 shows the architecture.

AOT Compiler. At skill installation time, the compiler analyzes the skill against the target environment and produces one or more optimized *skill variants*. The compiler performs three distinct passes: *Capability-based compilation* (§4.1) handles model and harness mismatch (P1–P2) by adapting the skill to the target’s available capabilities. *Environment binding* (§4.2) handles environment mismatch (P3) by materializing the dependencies required for execution. *Concurrency extraction* (§4.3) identifies parallelism latent in the skill’s workflow and maps it to the target harness. Compiled variants are stored per target.

Runtime & JIT optimizer. When a task arrives, the runtime selects the variant compiled for the current (model, harness) pair and loads it through the standard progressive disclosure mechanism. Two components then operate on top of normal execution. The *JIT optimizer* (§5.2) monitors execution outcomes across invocations and triggers recompilation when it detects capability gaps that the AOT compiler missed. It also compiles structurally fixed code patterns into executable functions that bypass the LLM inference entirely.

The *resource-aware scheduler* (§5.3) bridges compile-time parallelism annotations and run-time resource availability, throttling or suspending concurrent sub-agents when demand exceeds capacity.

4 Skill Compilation

When a user installs a skill for the first time, SkVM compiles it against the target (model, harness, host environment) before the skill ever runs. The compiler reads the raw skill text, identifies the target, and produces one or more optimized *skill variants* through three sequential passes. Each pass closes a different kind of gap between what the skill expects and what the target provides.

4.1 Capability-Based Compilation

The first pass addresses the model and harness mismatch. Because targets vary widely (§2.5), the compiler cannot embed optimization logic specific to each target. Instead, SkVM introduces *primitive capabilities*, an abstract vocabulary for expressing what a skill requires and what a target can provide. The compiler profiles each target offline with microbenchmarks for these capabilities, compares the profiling result with the skill’s capability requirements, and selects the optimization strategy based on the gap.

4.1.1 Primitive Capabilities

Just as JVM bytecodes define the basic operations a Java program requires from its runtime, *primitive capabilities* define the basic abilities a skill requires from its target. A primitive capability is an indivisible unit that describes what is needed to complete the tasks a skill defines. It is defined purely from the demand side, independent of any specific model. The definition of primitive capabilities follows three principles:

- *Composability*: any concrete task within a skill can be described as a combination of primitive capabilities.
- *Generality*: each primitive capability must be broad enough to appear across many skills, keeping the total capability set small and the compiler’s workload bounded.
- *Semantic independence*: a primitive capability concerns structural correctness, rather than whether the output is insightful or well-argued.

We derived the primitive capability set from a corpus of 15,063 skills in two stages. First, we curated 50 representative skills spanning document generation, script execution, data analysis, and code development, and used LLM-assisted analysis guided by the three principles above to decompose them into an initial set of 19 primitive capabilities. We then manually reviewed each primitive capability to verify that it satisfies all three principles. Second, we validated the initial set against the full corpus of 15,063 skills by checking whether each skill’s requirements can be expressed as a combination of the existing primitives. Skills that could not be covered revealed missing capabilities. A missing capability

Table 1. Representative primitive capabilities and their proficiency levels. The full catalog contains 26 primitive capabilities across four categories.

Primitive	L1	L2	L3
gen.code.shell	Basic commands (ls, cat)	Pipes, redirection, loops	Complex pipelines (sed, awk)
reason.arithmetic	Single-step ops	Multi-step	Compound
tool.exec	Single command	Params and relative paths	Chained multi-step execution
follow.procedure	3 sequential steps	5–7 with branches	Loops and verification

was added to the set only when it appeared in at least 1% of the corpus, filtering out rare or domain-specific requirements. This process converged on 26 primitive capabilities across four categories that cover the requirements of 95% of the skills.

We also observe that different tasks demand the same primitive capability at different depths. We therefore define multiple proficiency levels for each primitive capability. A higher level represents deeper use of the capability: an $L(n+1)$ requirement cannot be satisfied by $L(n)$ ability. Table 1 shows representative examples. For each primitive capability level, we generate a suite of microbenchmarks and evaluate them across different models. If a model succeeds on higher-level microbenchmarks while failing on lower-level ones, we interpret this inconsistency as evidence that the level hierarchy is miscalibrated. We then revise the level definitions and repeat the evaluation.

4.1.2 Measuring the Gap Between Skill and Target

Before applying optimizations, the compiler must determine the gap between what the skill requires and what the target provides. On the skill side, the compiler extracts the skill’s capability requirements at install time: it decomposes the skill into concrete tasks and maps each task to the primitive capabilities it requires.

On the target side, SkVM profiles the target with microbenchmarks generated for each primitive capability to measure how well the target supports it. Success on a microbenchmark for a primitive capability at a given level indicates that the model has attained at least that level of proficiency for that capability. This profiling is performed when the user sets up the agent and model. The results are cached and reused for all skill compilations for the same target.

4.1.3 Capability-Aware Skill Transforms

With the skill’s capability requirements and target’s capability profile in hand, the compiler walks through each capability requirement and selects a transform based on the gap between the required level and the target’s profiled level.

Compensation applies when the target supports the required capability but at a lower level. The compiler first identifies the distinctions between the two levels of a primitive capability. It then transforms the skill accordingly to reduce its capability requirement from a higher level to one supported by the target model. Such transformations may

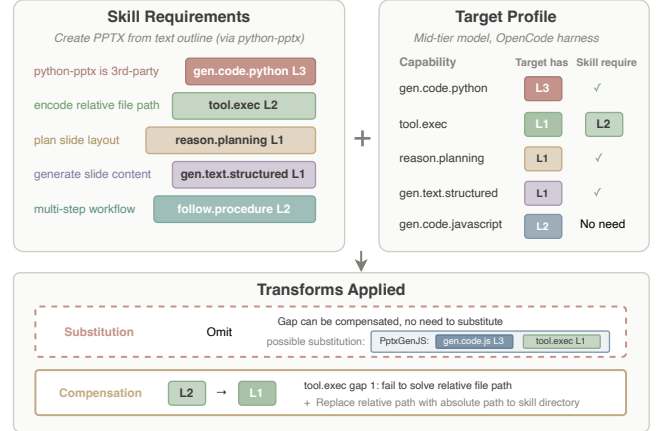


Figure 6. Capability-based compilation on a PPTX skill. The compiler extracts skill requirements (left), profiles the target (middle), and applies transforms based on the gap (bottom).

include providing examples, making instructions more explicit, and strengthening task constraints. Compensation is the preferred transform because it preserves the skill’s original intent.

Substitution is the fallback when compensation is not viable: the target lacks the capability entirely, or the level gap is too large for compensation to bridge reliably. The compiler switches to an alternative implementation path that achieves the same goal using different capabilities. For example, if a skill requires `gen.code.python` at L3 for a pandas workflow but the target only reaches L1, the compiler can switch to an SQL-based path if the target’s `gen.code.sql` reaches L2. These alternative paths form *equivalence classes* that the compiler identifies during requirement extraction.

Case Study. Figure 6 illustrates the compilation process on a real skill that teaches an agent to create PPTX presentations using `python-pptx`. The skill requires five primitive capabilities at varying levels. The target matches four of them directly but provides `tool.exec` at only L1, while the skill requires L2 for resolving relative file paths.

The compiler identifies a possible substitution path: switching to `PptxGenJS` via `gen.code.javascript` would replace `python-pptx` and lower the `tool.exec` requirement to L1. However, because the `tool.exec` gap is only one level, compensation is preferred. The L2 profiling results reveal that this model fails to resolve relative file paths when executing commands. The compiler injects absolute path resolution into the compiled skill, replacing relative paths with absolute paths to the skill directory.

4.2 Environment Binding

The second pass resolves *P3: environment mismatch*. Rather than merely checking whether dependencies are installed, the compiler uses a built-in *env-binding skill* to reconcile

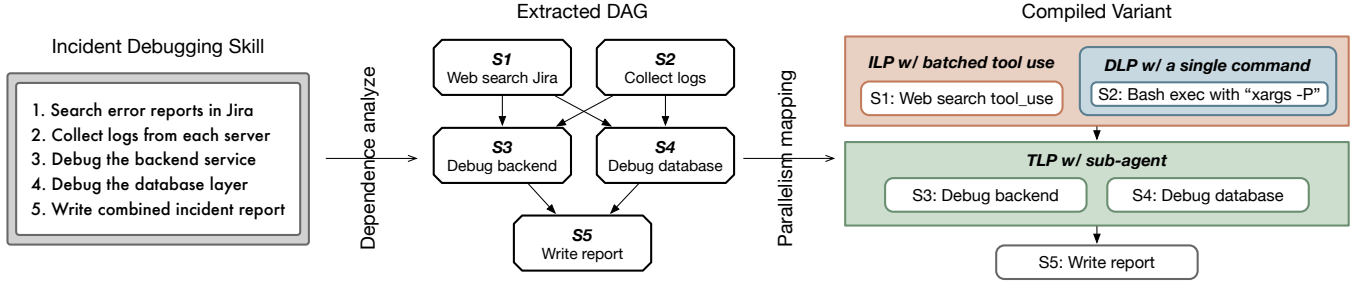


Figure 7. Concurrency extraction on an incident-response skill. The compiler decomposes sequential steps into a workflow DAG, identifies parallel opportunities both between steps and within steps, and maps each to the appropriate concurrency primitive.

the skill’s assumptions against the host environment and generate an env-binding script.

At skill installation time, the compiler first extracts a dependency manifest from the output skill of the first pass and any explicit prerequisites, covering external libraries, CLI tools, and system services. For each manifest entry, the compiler runs a lightweight presence check (e.g., `pip show`, which). Dependencies that are already satisfied require no further work. When the check fails, the compiler performs deeper, system-aware probing to determine how to resolve the dependency. The compiler then emits an idempotent env-binding script which will check and repair the dependency before skill execution. When the environment changes or the env-binding script fails, execution falls back to the model, and the script’s output is passed to the model as additional context.

4.3 Concurrency Extraction

The third pass finds parallelism hiding in plain sight. Recall that 76% of skills contain procedural structure (§2.3). These skills describe multi-step workflows in sequential prose, but not every step actually depends on the one before it.

The compiler uses LLM-assisted analysis to decompose the skill into discrete steps, identifying for each step what inputs it consumes and what outputs it produces. It then builds a workflow DAG: nodes are steps, and an edge from step A to step B exists when B consumes an output of A. The compiler also analyzes each step internally for sub-operations that can proceed concurrently. Figure 7 illustrates this process on an incident-response skill. For each parallel opportunity, whether between steps or within a step, the compiler selects a concurrency primitive based on the task structure and the harness. SkVM defines three levels of parallelism.

Data-Level Parallelism (DLP). A single step applies the same operation to multiple independent data items, such as running the same analysis on each of 15 CSV files. The compiler rewrites the step to process items concurrently using language-level parallelism primitives appropriate to the skill’s implementation, such as shell-level parallel execution,

Python’s multiprocessing, or JavaScript’s Promise.all. DLP requires no special harness support.

Instruction-Level Parallelism (ILP). Multiple independent steps each need a tool call with no data dependency between them, such as running eight independent code-analysis scripts on a project. The compiler groups the corresponding DAG nodes into one parallel stage and rewrites the skill so that their tool invocations are issued together in a single LLM turn, instead of one after another in sequential prose. The compiler annotates the skill with which step outputs should be bound back to which downstream DAG inputs after the batch completes. ILP requires the harness to support batch tool dispatch.

Thread-Level Parallelism (TLP). The workflow decomposes into independent sub-tasks that each require multi-turn reasoning, such as debugging three independent services. The compiler rewrites the skill to extract each sub-task into a separate sub-agent block with an explicit task description, declared input context, and expected output that can be consumed by the parent workflow. The annotation marks this block for sub-agent dispatch, so the runtime spawns one sub-agent per block in its own session and merges the returned outputs back into the DAG. TLP requires harness support for sub-agent spawning.

Opportunities whose required primitive is unavailable on the target harness fall back to sequential execution. For each viable opportunity, the compiler either rewrites the skill’s instructions to execute items concurrently (DLP) or emits execution annotations that make the parallel structure explicit: a parallel stage over DAG nodes for ILP or a sub-agent task block with input and output contracts for TLP. The runtime reads these annotations and dispatches them with the corresponding harness primitive.

5 Skill Runtime with JIT Optimization

AOT compilation generates multiple skill variants for different targets. Consequently, at runtime, SkVM loads the skill variant compiled for the current target. Since AOT compilation is performed only once per skill and target (model+harness) pair, the compilation overhead is modest.

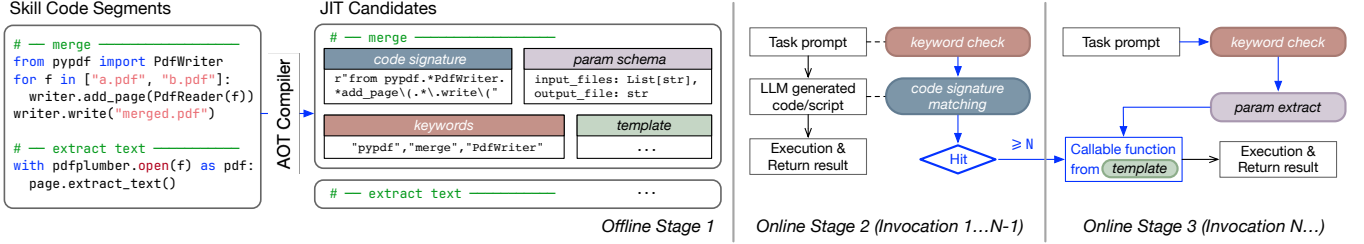


Figure 8. Code solidification pipeline. The AOT compiler analyzes skill code segments and generates JIT candidates with code signatures and templates (Stage 1). The runtime validates predictions across invocations (Stage 2). After promotion, a compiled function replaces LLM inference (Stage 3).

However, some problems only surface at execution time, such as capability gaps that static profiling missed or resource contention among parallel sub-agents competing for rate-limited APIs. Other opportunities, like code patterns that recur across invocations, only become visible after repeated runs. The SkVM runtime addresses both through JIT optimization (§5.2) and a resource-aware scheduler (§5.3).

5.1 Skill Registration and Execution

Compilation produces multiple skill variants for each skill, along with the env-binding scripts generated by Pass 2. When a new task arrives, the runtime selects the variant compiled for the current (model, harness) pair. From the agent’s perspective, the complexity of multiple variants is invisible: it sees a single set of skills and loads them through the same progressive disclosure mechanism described in §2. Before loading the full skill context, the runtime will first execute the env-binding script to verify whether runtime dependencies are satisfied.

5.2 JIT Optimization

Beyond AOT compilation, which tailors primitive capabilities to each target, SkVM further applies JIT optimization to enhance execution quality and efficiency through two complementary mechanisms: adaptive recompilation and code solidification.

5.2.1 Adaptive Recompilation

The runtime tracks the outcome of every task executed with a skill. When a skill execution fails (e.g., agent abort, human feedback) or retries during the agent loop, the runtime records a structured failure or retry log. Although the current model may perform self-correction through reflection, the correction process is not recorded, leading to the possibility of encountering the same errors in subsequent executions.

When the same skill fails across multiple invocations, the runtime analyzes whether the failures are task-specific or reflect a systematic capability gap in the skill itself. Only in the latter case does the runtime trigger adaptive recompilation: it feeds the accumulated failure logs and the model’s self-recovery traces as input to the compiler, which then applies targeted compensation transforms to the skill. If the

recompiled variant performs worse, the runtime rolls back to the previous version. Each subsequent recompilation starts from the best-performing variant observed so far, ensuring that optimization is improving skill performance.

5.2.2 Code Solidification

Recall that 75% of skills contain solidifiable code (§2.3): code whose overall structure is fixed, varying only in input-specific parameters. In normal execution, the LLM re-executes the full reasoning and tool-use cycle on every invocation, spending tokens and latency on a process whose structure is identical each time. Code solidification identifies these repetitive patterns and promotes them into executable code that bypasses model inference. Figure 8 illustrates the three-stage pipeline.

In the first stage, the AOT compiler analyzes code and script segments within each skill, identifying whether they contain parameterized patterns eligible for solidification. For eligible segments, it generates a *JIT candidate* containing four components: keywords that filter relevant invocations, a *code signature* that captures the expected output structure as a match pattern, a code template with parameter slots, and a parameter schema describing the extractable arguments. These candidates are bundled with the compiled skill variant.

In the second stage, the runtime monitor watches LLM invocations across repeated calls. For each invocation, it first checks keywords for relevance, then matches the generated code against the code signature. The code solidification is triggered only after the code signature matches successfully across multiple consecutive invocations, ensuring that the code structure generated by the LLM remains consistent with the patterns analyzed in the AOT stage. If the model’s output consistently diverges from the code signature, the system stays on the safe LLM path and never promotes.

In the third stage, the template is instantiated into a callable function: the template code is wrapped with parameter extraction and output handling to produce a standalone shell script or code function. This instantiation step is lightweight and mechanical, as the AOT stage has already done the analysis. Subsequent invocations bypass the LLM entirely: the runtime extracts parameters from the task context, calls the instantiated function, and returns the result. If the solidified

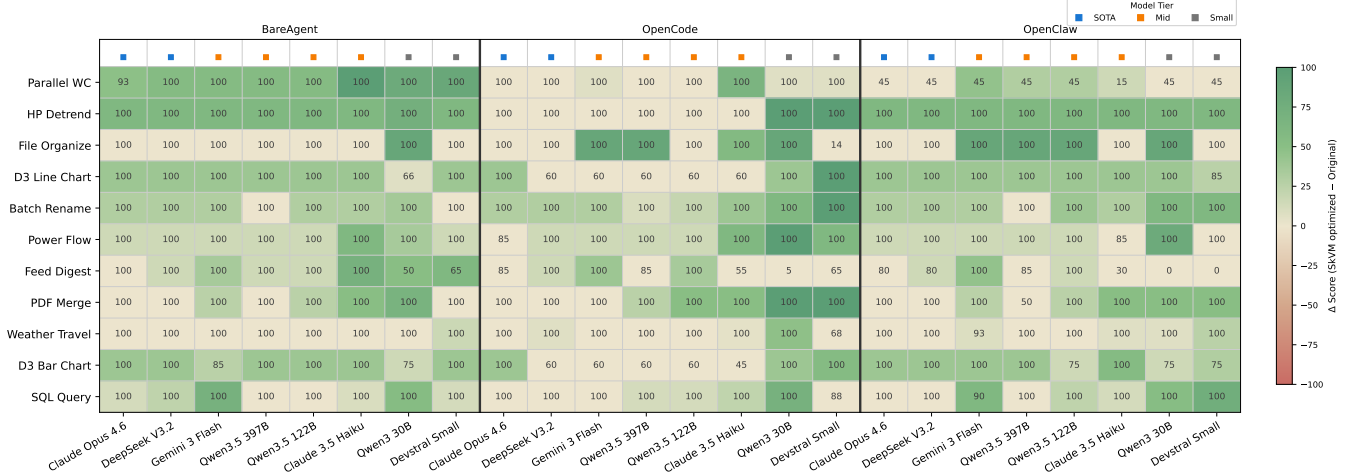


Figure 9. Skill compilation effectiveness. Cell values represent task completion rates after SkVM optimization, while cell color indicates the score improvement (green) or regression (red) relative to original baseline skills.

code execution causes an agent task to fail or encounters a runtime exception, SkVM triggers a fallback mechanism that re-enables LLM-based code generation to ensure correctness.

5.3 Resource-Aware Parallel Scheduling

In Pass 3 of AOT compilation, the compiler identifies where parallelism is possible, but how much parallelism is *efficient* depends on conditions that change at run time: CPU usage, external API rate limit, the machine’s available memory, and other shared resources.

SkVM dynamically monitors system state and schedules task execution accordingly, bridging the gap between compile-time opportunity and run-time reality. For API-bound workloads, it watches response latencies and HTTP 429 (rate limit) signals. For compute-bound workloads, it monitors CPU and memory usage. When pressure rises above a threshold, SkVM applies two mechanisms to mitigate resource contention. First, it throttles the launch of new sub-agents to avoid introducing additional concurrency pressure. Second, for sub-agents that are already running, SkVM selectively suspends a subset of them (i.e., pauses their agent-loop execution), based on per-agent resource usage or launch order, to reduce active competition for shared resources. SkVM also records the effective concurrency of each skill from its previous run on the current system and uses it as a concurrency hint for subsequent executions.

6 Evaluation

6.1 Setup

Benchmark. We select and extend existing skill benchmarks, including SkillsBench [30] and PinchBench [40], covering diverse agent tasks spanning code generation, data analysis, document creation, and system administration. Each task is paired with at least one skill sourced from mainstream skill sources: Anthropic-skills [6], OpenClaw skills [37], and

skills.sh [50]. We employ an automated evaluation suite comprising static analysis and LLM-based assessment against predefined criteria [26, 59].

Models. We evaluate SkVM across eight models spanning three capability tiers to comprehensively assess its effectiveness across varying model capacities. *SOTA*: claude-opus-4.6 [10], deepseek-v3.2 [31]. *Mid-tier*: gemini-3-flash [17], qwen3.5-397b [44], qwen3.5-122b [43], claude-3.5-haiku [4]. *Small*: qwen3-30b [42], devstral-small [34].

Baselines. We compare SkVM against three baselines. *No Skill*: the agent executes without any skill loaded. *Original*: the original skill. *Skill-Creator*: the skill optimized by Anthropic’s Skill-Creator [9] using claude-opus-4.6.

Harnesses. We select three different open-source agent harnesses to evaluate SkVM performance across diverse execution environments. Each harness offers distinct functionality: *BareAgent*: a minimal harness that injects the compiled skill directly into the system prompt without additional abstraction layers. *OpenCode* [38]: a full-featured code agent harness with batch tool dispatch capabilities and a rich tool set for code execution. *OpenClaw* [37]: a general-purpose agent harness with integrated functional modules designed to support complex multi-step task execution.

Methodology. We first profile the primitive capabilities of different models, and then apply SkVM’s AOT and JIT compilation to generate optimized skill variants tailored to each model. To support the compiled skill artifacts, we extend the agent harnesses with custom loading and management mechanisms that enable SkVM’s optimizations. For each task, we generate five diverse input instances and measure the average task completion rate, token consumption, and end-to-end execution latency. All experiments are conducted on a Mac Mini M4 with 16GB memory.

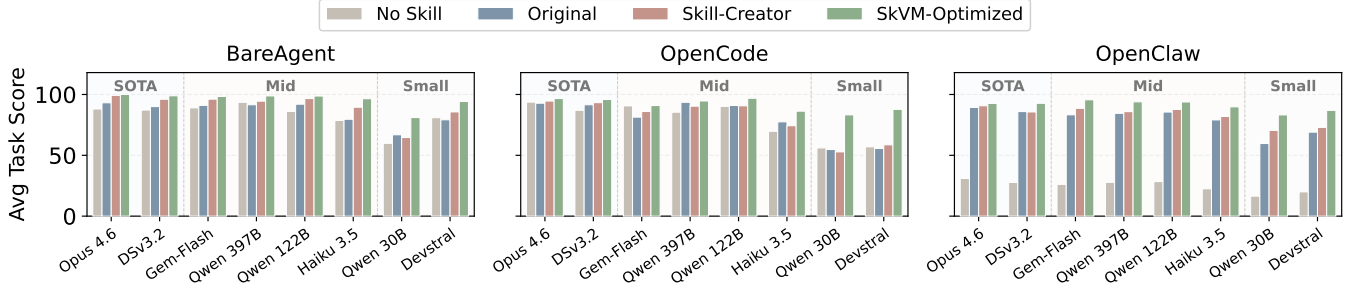


Figure 10. Average task score by skill variant. Four variants are compared across eight models and three harnesses: No Skill, Original, Skill-Creator, and SkVM-Optimized. Tier boundaries (SOTA / Mid / Small) are shaded. SkVM-Optimized skills consistently achieve the highest scores, with the largest absolute gains for weaker models and on OpenClaw.

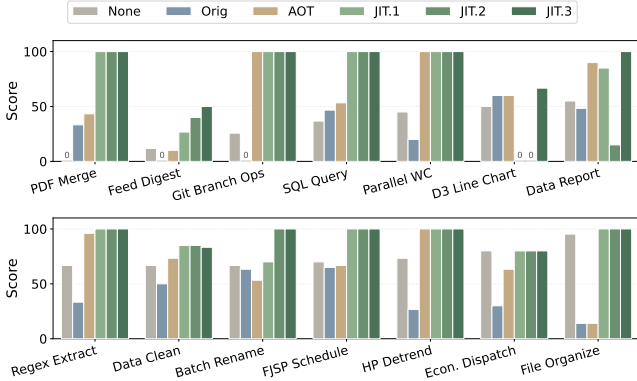


Figure 11. Staged optimization breakdown across 14 skill categories. Each group shows scores at six stages: no skill, original skill, AOT-compiled skill, and skills optimized in three JIT rounds.

6.2 Skill Compilation Effectiveness

We first evaluate SkVM’s effectiveness in improving task completion rates. We compare SkVM-optimized skills against unoptimized baseline skills across different tasks, models, and agent harnesses. For SkVM, the primary gains stem from capability-based compilation and JIT optimization. Figure 9 presents compilation effectiveness across eight models and three harnesses. Cell values represent task completion rates after SkVM optimization, while cell color indicates the score improvement (green) or regression (red) relative to unoptimized baseline skills.

Figure 9 demonstrates SkVM’s optimization effectiveness (compared with original skills) across diverse models and agent harnesses. Weaker models benefit more substantially from SkVM optimization. This finding indicates that for most agent tasks, weaker models possess sufficient capability for the logical components but lack proficiency in non-logical aspects, such as generating complex JSON structures or managing environment dependencies. SkVM’s compilation optimization addresses these gaps through primitive capability refinement, thereby improving overall task completion rates. For stronger models, improvements are more

modest, as these models already handle tasks competently; SkVM optimization primarily reduces token consumption and execution latency. Figure 10 aggregates average task scores across all four skill variants for each model and harness. SkVM-optimized skills achieve the highest scores on every model-harness combination. The improvement over the Skill-Creator baseline grows with decreasing model capability: on BareAgent, SkVM-optimized skills improve over Skill-Creator by 25% for qwen-30b and 10% for devstral-small. In terms of regressions, SkVM-compiled skills reduce task completion rates on only 4.5% of tasks, compared with 15% for the original skills. Using the original skill, OpenCode and OpenClaw differ by up to 13 points, depending on the model, which shows that harness behavior alone materially changes outcomes. SkVM optimization raises scores on both harnesses and reduces this cross-harness gap to at most 5 points.

6.3 Staged Optimization Breakdown

Figure 11 breaks down the contribution of each optimization stage across 14 skill categories on Qwen3-30B with BareAgent. Each group shows scores at six stages: no skill, original skill, AOT-compiled skill, and up to three rounds of JIT optimization. Tasks used in each round are of the same type but differ in contents.

In 11 of 14 categories, the original skill performs worse than using no skill. After SkVM’s AOT compilation, the average task score improves by 88%. JIT optimization identifies additional capability defects that were not exposed during AOT compilation, further improving performance. After one JIT round, 8 of 14 skills achieve full scores on the evaluation tasks, and after three rounds, 10 of 14 do so.

JIT optimization can introduce regressions. For example, in the Line Chart task, the JIT compiler added an example file to the skill, but the file was too long and triggered parsing errors, which reduced the score. After three rounds of revision guided by prior failure logs, the JIT compiler eventually fixed this issue.

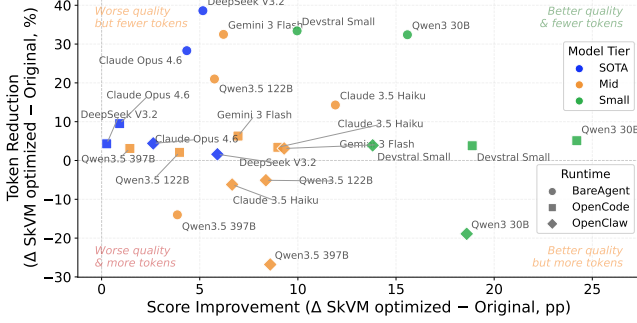


Figure 12. Score gain vs. token savings for SkVM optimization over the Original baseline. Each point is a (model, harness) pair. Color = model tier, shape = harness. Most points land in the upper-right quadrant: better quality *and* fewer tokens.

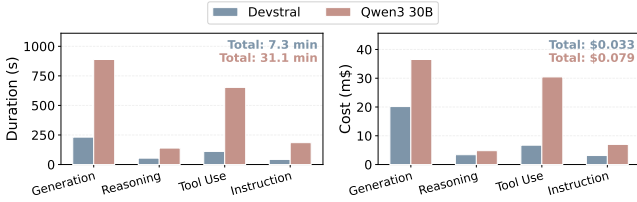


Figure 13. Per-category breakdown of capability profiling overhead for two small models. Left: duration in seconds. Right: cost in millidollars.

6.4 Token and Cost Efficiency with Compilation

Beyond improving task success rates, SkVM provides substantial cost benefits for SOTA LLMs by reducing token consumption. LLMs can correct execution errors through iterative agent loops, refining actions based on environment feedback. Therefore, even when tasks eventually succeed, this approach may incur substantial token waste. Figure 12 shows how task success rates and token costs vary across different model tiers and harnesses after SkVM compilation.

Our evaluation reveals that for most models and harness combinations, SkVM compilation simultaneously improves task success and reduces token consumption. For the strongest model and weakest harness pairing (DS-v3.2 + BareAgent), we observe token savings approaching 40%. This improvement arises because agent loops incur interaction overhead when correcting environment-related and tool-invocation errors; multiple retries are often required. By employing JIT compilation, SkVM enables models to avoid these error paths entirely, eliminating redundant interactions and saving substantial tokens.

For weaker models, SkVM may occasionally consume additional tokens. This occurs because SkVM improves task completion rates, leading to longer agent execution traces compared to the baseline skill, resulting in modest token increases in edge cases.

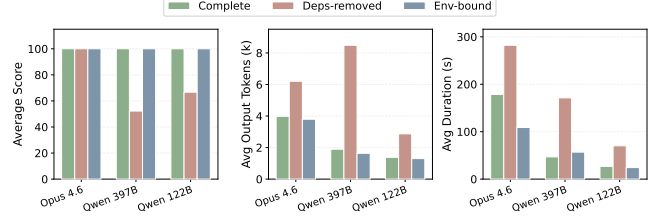


Figure 14. Environment binding restores correctness, token efficiency, and execution speed. From left to right: average score, average token use, and average duration across two tasks per model. Env-bound performance returns to complete-environment levels for all three models.

6.5 Overhead of Target Profiling

Capability-based compilation requires profiling the target once to build its capability profile (§4.1). This profiling is a one-time cost per (model, harness) pair, and the results are cached and reused for all subsequent skill compilations on the same target. We measure this overhead on two small models, devstral-small and qwen3-30b.

Figure 13 shows the per-category breakdown of profiling duration and cost. A full profile across all primitive capabilities completes in 7.3 minutes for devstral-small and 31.1 minutes for qwen3-30b, costing \$0.033 and \$0.079 respectively.

6.6 Evaluating Environment Binding

The preceding two experiments conducted testing in fully provisioned environments. However, in real-world skill scenarios, missing environment dependencies during agent execution are commonplace. Resolving such environmental issues presents significant challenges for weaker LLMs, frequently resulting in skill execution failures. SkVM addresses this through an environment binding mechanism in its AOT compilation phase. This mechanism shifts environment dependency installation from the LLM execution phase to pre-execution preprocessing, allowing the LLM to focus solely on skill logic.

Figure 14 demonstrates the performance of different models across three environmental configurations: complete, with environment binding, and with missing dependencies. Stronger models such as claude-opus-4.6 can autonomously recover from missing dependencies, but at the cost of increased token consumption. For weaker models such as qwen3.5-122b, missing dependencies lead to execution failures. Environment binding fully restores execution correctness and substantially reduces token consumption.

6.7 Evaluating Concurrency Extraction

SkVM extracts more parallelism opportunities within skills and improves efficiency. We evaluate three parallelism types: data-level parallelism (DLP), instruction-level parallelism (ILP), and thread-level parallelism (TLP). DLP applies when

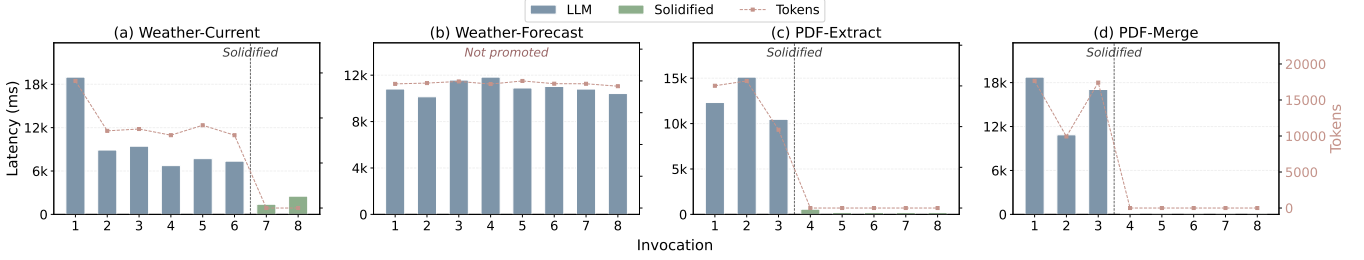


Figure 15. Code solidification across four cases. Blue bars show LLM inference latency, green bars show solidified execution. The dashed line marks the promotion point. Weather-forecast never promotes because the runtime model’s code patterns diverge from the AOT prediction, validating the promotion gate as a safety mechanism.

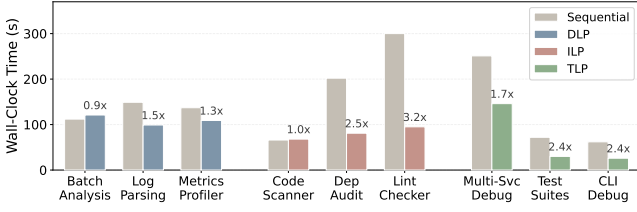


Figure 16. Performance of different parallelization strategies across eight tasks. Bars show sequential, DLP, ILP, and TLP execution.

instructions within a single skill batch-process large amounts of independent data. ILP applies when multiple independent instructions or code segments within a skill can execute concurrently. TLP applies when multiple independent sub-agents run within a skill, each processing self-contained sub-tasks without cross-dependencies. Figure 16 demonstrates SkVM’s performance improvements across these three parallelism types.

Experimental results show that SkVM’s parallelism extraction strategy achieves up to 3.2× end-to-end speedup. TLP yields the largest average improvements because its coarser-grain parallelism produces more pronounced optimization effects. ILP, operating at the instruction level, reduces both execution time and the number of LLM invocations. DLP gains stem purely from data parallelism, such that the performance improvement scales directly with the degree of data parallelism. We also observe inherent variability in agent execution due to LLM throughput fluctuations and varying numbers of agent loop iterations. Consequently, timing variations within a reasonable range are acceptable.

6.8 Evaluating Code Solidification

We further analyze whether adopting code solidification mechanisms from JIT optimization can accurately identify reusable code segments and improve agent execution efficiency. As shown in Figure 15, we conduct a detailed analysis on two canonical skills (weather, document-pdf). For the two PDF tasks, despite variations in task inputs, the LLM-generated code matches the code signature. Therefore, SkVM can apply JIT compilation optimization, directly generating

solidified code using the template and input parameters. For PDF-extract tasks, JIT optimization reduces execution time from 10,469–15,116 ms to 206–568 ms, achieving a 19–50× speedup. For the two weather tasks, weather-current requires external weather API calls, the speedup is bounded by network latency, reducing execution time from 9,000 ms to 2,000 ms, yielding a 5× improvement. In contrast, weather-forecast uses more flexible formats, so the generated code structures diverge from the code signature and fail to match. Consequently, all eight invocations rely on LLM generation to avoid invoking incorrect code.

If solidified code execution causes task failure, SkVM’s safety mechanism triggers fallback, re-enabling LLM code generation to ensure correctness.

7 Discussion

Non-determinism in skill compilation. Unlike traditional program compilation, skill compilation takes natural language as input, which inherently introduces some non-determinism into the compilation process [55]. However, the LLM executing skills differs fundamentally from traditional hardware such as CPUs [23], and possesses an inherent tolerance for input variability that enables skills to execute correctly. Moreover, SkVM’s rigorous compilation optimization passes and rollback mechanisms ensure that compiled skills consistently deliver stable performance improvements across most downstream tasks.

Capability Coverage. The current catalog has 26 primitive capabilities across four categories, enough to cover the 15,063 skills we analyzed. As the skill library grows, we can adopt the iterative approach described in §4.1 to expand the primitive capabilities.

Compilation cost. AOT compilation invokes an LLM, which incurs token costs. However, since skills are executed repeatedly at runtime, the compilation overhead is amortized across multiple invocations [21]. Furthermore, compiled skills can be shared across users and applications, further reducing the per-skill compilation cost.

8 Conclusion

Skills have emerged as a new code form in the agent era. However, after analyzing over 100,000 skills, we discover a significant mismatch between skills and underlying LLMs. To address this, we propose SkVM, a compilation and run-time system specifically tailored for skills. SkVM leverages compilation techniques such as AOT and JIT to optimize skill structure, extract environment dependencies, and identify parallelism opportunities. As models and agent harnesses continue to diversify, skill compilation provides a systematic pathway to transform skills into portable components, rather than fragile prompts.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Agent Skills Initiative. Agent skills specification. <https://agentskills.io/specification>, 2025. Open SKILL.md format for agent skill portability; accessed: 2026-02-06.
- [3] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2006.
- [4] Anthropic. Claude 3.5 haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>, 2024. Accessed: 2026-04-02.
- [5] Anthropic. Agent skills. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>, 2025. Accessed: 2026-02-02.
- [6] Anthropic. Anthropic skills repository. <https://github.com/anthropics/skills>, 2025. Accessed: 2026-03-22.
- [7] Anthropic. Equipping agents for the real world with agent skills. <https://claude.com/blog/equipping-agents-for-the-real-world-with-agent-skills>, 2025. Accessed: 2026-02-06.
- [8] Anthropic. Extend claude with skills. <https://code.claude.com/docs/en/skills>, 2025. Claude Code skill mechanism; accessed: 2026-02-06.
- [9] Anthropic. Skill creator. <https://github.com/anthropics/skills/tree/main/skills/skill-creator>, 2025. Accessed: 2026-03-23.
- [10] Anthropic. Claude 4.6 opus. <https://www.anthropic.com/claude/opus>, 2026. Accessed: 2026-04-02.
- [11] Anthropic. Claude code. <https://github.com/anthropics/claude-code>, 2026. Accessed: 2026-04-01.
- [12] John Aycock. A brief history of just-in-time. *ACM computing surveys (CSUR)*, 35(2):97–113, 2003.
- [13] ClawHub. Clawhub: Agent skills marketplace. <https://clawhub.ai>, 2025. Accessed: 2026-03-22.
- [14] Coze. Plugin development guide. https://www.coze.com/docs/guides/plugin?_lang=en, 2025. Coze bot plugins and skills; accessed: 2026-02-06.
- [15] Timothy Cramer, Richard Friedman, Terrence Miller, David Seberger, Robert Wilson, and Mario Wolczko. Compiling java just in time. *Ieee micro*, 17(3):36–43, 1997.
- [16] Cursor. Agent skills. <https://cursor.com/docs/context/skills>, 2025. Cursor IDE skill mechanism; accessed: 2026-02-06.
- [17] DeepMind. Our latest gemini 3 model that helps you bring any idea to life - faster. <https://deepmind.google/models/gemini/flash/>, 2026. Accessed: 2026-04-02.
- [18] FreedomIntelligence. Openclaw medical skills. <https://github.com/FreedomIntelligence/OpenClaw-Medical-Skills>, 2025. Accessed: 2026-03-23.
- [19] Google. Agent skills. <https://antigravity.google/docs/skills>, 2025. Google Antigravity coding agent skills; accessed: 2026-02-06.
- [20] Google. Agent2agent (a2a) protocol. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>, 2025. Accessed: 2026-02-06.
- [21] Jim Gray. The transaction concept: Virtues and limitations. *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, pages 144–154, 1981.
- [22] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. Swe-skills-bench: Do agent skills actually help in real-world software engineering? *arXiv preprint arXiv:2603.15401*, 2026.
- [23] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2019.
- [24] Kazuaki Ishizaki, Motohiro Kawahito, Toshiaki Yasue, Hideaki Komatsu, and Toshio Nakatani. A study of devirtualization techniques for a java just-in-time compiler. In *Proceedings of the 15th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*, pages 294–310, 2000.
- [25] Jan Martin Jansen, Pieter Koopman, and Rinus Plasmeijer. From interpretation to compilation. In *Central European Functional Programming School*, pages 286–301. Springer, 2007.
- [26] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [27] Sajith Kalathingal, Caroline Collange, Bharath N Swamy, and André Seznec. Dynamic inter-thread vectorization architecture: extracting dlp from tlp. In *2016 28th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, pages 18–25. IEEE, 2016.
- [28] Daniel Kästner and Sebastian Winkel. Ilp-based instruction scheduling for ia-64. *ACM SIGPLAN Notices*, 36(8):145–154, 2001.
- [29] LangChain. Langgraph: Build resilient language agents as graphs. <https://github.com/langchain-ai/langgraph>, 2024. Accessed: 2026-03-22.
- [30] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026.
- [31] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- [32] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023. arXiv:2107.13586, 2021.
- [33] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations (ICLR)*, 2024.
- [34] Mistral AI. Devstral small 2507. <https://huggingface.co/mistralai/Devstral-Small-2507>, 2026. Accessed: 2026-04-02.
- [35] OpenAI. Function calling and other api updates. <https://openai.com/index/function-calling-and-other-api-updates/>, 2023. Accessed: 2026-02-06.
- [36] OpenAI. Openai codex. <https://developers.openai.com/codex>, 2025. Accessed: 2026-02-06.
- [37] OpenClaw. Openclaw: Personal ai assistant. <https://github.com/openclaw/openclaw>, 2025. Accessed: 2026-03-22.
- [38] OpenCode. Opencode: Open-source ai coding agent. <https://github.com/opencode-ai/opencode>, 2025. Accessed: 2026-03-22.
- [39] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions

- with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 27730–27744, 2022.
- [40] PinchBench. Pinchbench: Benchmarking llm models as coding agents. <https://github.com/pinchbench/skill>, 2026. Accessed: 2026-02-06.
 - [41] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations (ICLR)*, 2024.
 - [42] Qwen. Qwen-3-30b. <https://huggingface.co/Qwen/Qwen3-30B-A3B>, 2026. Accessed: 2026-04-02.
 - [43] Qwen. Qwen-3.5-122b. <https://huggingface.co/Qwen/Qwen3.5-122B-A10B>, 2026. Accessed: 2026-04-02.
 - [44] Qwen. Qwen-3.5-397b. <https://huggingface.co/Qwen/Qwen3.5-397B-A17B>, 2026. Accessed: 2026-04-02.
 - [45] Narayan Ranganathan and Manoj Franklin. An empirical study of decentralized ilp execution models. *ACM SIGPLAN Notices*, 33(11):272–281, 1998.
 - [46] Joshua Redstone, Susan Eggers, and Henry Levy. Mini-threads: Increasing tlp on small-scale smt processors. In *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings.*, pages 19–30. IEEE, 2003.
 - [47] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
 - [48] Manuel Serrano. Javascript aot compilation. *ACM SIGPLAN Notices*, 53(8):50–63, 2018.
 - [49] Manuel Serrano. Of javascript aot compilation performance. *Proceedings of the ACM on Programming Languages*, 5(ICFP):1–30, 2021.
 - [50] Skills.sh. Skills.sh: Agent skills registry. <https://skills.sh>, 2025. Accessed: 2026-03-22.
 - [51] Bjarne Stroustrup. *The C++ programming language*. Pearson Education, 2013.
 - [52] Toshio Suganuma, Takeshi Ogasawara, Mikio Takeuchi, Toshiaki Yasue, Motohiro Kawahito, Kazuaki Ishizaki, Hideaki Komatsu, and Toshio Nakatani. Overview of the ibm java just-in-time compiler. *IBM systems Journal*, 39(1):175–193, 2000.
 - [53] Bill Venners. *Inside the Java Virtual Machine*. McGraw-Hill, 1998.
 - [54] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
 - [55] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, et al. Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations (ICLR)*, 2023.
 - [56] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. In *Conference on Language Modeling (COLM)*, 2024.
 - [57] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
 - [58] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023.
 - [59] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural*